CO3_AT2

COMPILER DESIGN – SEMANTIC ANALYSIS AND SYNTAX-DIRECTED DEFINITIONS

Q1. Semantic Analysis and Syntax Tree Construction

Given expression:

a + b * c - d

The important rules are:

- * has higher precedence than + and -.
- + and - have the same precedence.
- + and - are left associative.

Therefore:

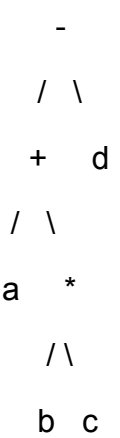
a + b * c - d

is evaluated as:

(a + (b * c)) - d

(i) Syntax Tree

The syntax tree is:



Therefore, the order of operations is:

1. b * c
2. a + (b * c)
3. (a + (b * c)) - d

(ii) Syntax-Directed Definition (SDD)

Use the following grammar:

E → E + T

E → E - T

E → T

T → T * F

T → F

F → id

Let E.val, T.val, and F.val be synthesized attributes.

Semantic Rules

E → E1 + T

E.val = E1.val + T.val

E → E1 - T

E.val = E1.val - T.val

E → T

E.val = T.val

T → T1 * F

T.val = T1.val * F.val

T → F

T.val = F.val

F → id

F.val = id.lexval

Here:

- lexval = value of the identifier.
- val = computed value of an expression.

This is an **S-attributed definition** because all attributes used for evaluation are synthesized attributes.

(iii) Attribute Evaluation

For:

a + b * c - d

First calculate:

b * c

Therefore:

T.val = b.val * c.val

Then:

a + (b*c)

So:

E.val = a.val + T.val

Finally:

(a + b*c) - d

Therefore:

E.val = previous_E.val - d.val

If:

a = 2

b = 3

c = 4

d = 5

then:

b*c = 3*4 = 12

a + b*c = 2 + 12 = 14

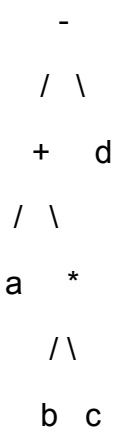
14 - d = 14 - 5 = 9

Final value:

E.val = 9

(iv) How the Syntax Tree Represents the Correct Order

The tree structure automatically represents precedence.



The * node is below the + node, showing that multiplication is performed first.

The root is -, so subtraction is performed after the addition.

Thus:

a + b*c - d

is correctly represented as:

(a + (b*c)) - d

The tree also represents **left associativity** because:

a + b*c

becomes the left operand of -.

Conclusion

The syntax tree and SDD together ensure:

- Correct precedence
- Correct associativity
- Correct expression evaluation
- Correct internal representation

Q2. S-Attributed SDD and Bottom-Up Evaluation

Given grammar

S → E n

E → E + T

E → E - T

E → T

T → T * F

T → T / F

T → F

F → (E)

F → digit

Input

8 + 2 * 4 n

(i) S-Attributed SDD

Let val be a synthesized attribute.

Semantic Rules

S → E n

S.val = E.val

E → E1 + T

E.val = E1.val + T.val

E → E1 - T

E.val = E1.val - T.val

E → T

E.val = T.val

T → T1 * F

T.val = T1.val * F.val

T → T1 / F

T.val = T1.val / F.val

T → F

T.val = F.val

F → (E)

F.val = E.val

F → digit

F.val = digit.lexval

All attributes are synthesized, so this is an **S-attributed SDD**.

(ii) Code Fragment / Translator

A simple translator can generate three-address code.

For the expression:

8 + 2 * 4

the generated intermediate code is:

t1 = 2 * 4

t2 = 8 + t1

Final result:

16

Simple C-style translator fragment

```
#include <stdio.h>

int main()
{
    int a = 8;
    int b = 2;
    int c = 4;

    int t1 = b * c;
    int t2 = a + t1;

    printf("t1 = %d\n", t1);
    printf("t2 = %d\n", t2);
    printf("Result = %d\n", t2);

    return 0;
}
```

Output

t1 = 8

t2 = 16

Result = 16

(iii) Parser Stack Demonstration

Input:

8 + 2 * 4 n

We use bottom-up parsing.

Step 1 – Shift 8

Stack: 8

Input: + 2 * 4 n

Action: Shift 8

Reduce:

F → digit

F.val = 8

Then:

T → F

T.val = 8

Then:

E → T

E.val = 8

Step 2 – Shift +

Stack: E +

Input: 2 * 4 n

Action: Shift +

Step 3 – Shift 2

Stack: E + 2

Input: * 4 n

Action: Shift 2

Reduce:

F → digit

F.val = 2

Then:

T → F

T.val = 2

Step 4 – Shift *

Stack: E + T *

Input: 4 n

Action: Shift *

Because * has higher precedence, multiplication must be completed before addition.

Step 5 – Shift 4

Stack: E + T * 4

Input: n

Action: Shift 4

Reduce:

F → digit

F.val = 4

Now reduce:

T → T * F

Therefore:

T.val = 2 * 4

= 8

Intermediate code:

t1 = 2 * 4

Step 6 – Addition

Now reduce:

E → E + T

Therefore:

E.val = 8 + 8

= 16

Intermediate code:

t2 = 8 + t1

Step 7 – Final Reduction

S → E n

Therefore:

S.val = 16

Complete Table

Step	Stack	Input	Action	Value
1	8	+ 2 * 4 n	Shift 8	8
2	E	+ 2 * 4 n	Reduce F→digit	8
3	E +	2 * 4 n	Shift +	—
4	E + 2	* 4 n	Shift 2	2
5	E + T	* 4 n	Reduce T→F	2
6	E + T *	4 n	Shift *	—
7	E + T * 4	n	Shift 4	4
8	E + T * F	n	Reduce F→digit	4
9	E + T	n	Reduce T→T*F	8
10	E	n	Reduce E→E+T	16

Step	Stack	Input	Action	Value
11	S	—	Reduce S→En	16

(iv) Syntax Tree with Computed Values

Expression:

8 + 2 * 4

Syntax tree:

```
          +
        /   \
       8     *
           / \
          2   4
```

Annotated tree:

```
          + [16]
        /       \
     8 [8]       * [8]
               /     \
            2 [2]   4 [4]
```

Therefore:

2 * 4 = 8

8 + 8 = 16

Final value:

16

(v) Why Bottom-Up SDD Gives Correct Precedence

The grammar separates expressions into different levels:

E → E + T | E - T | T

T → T * F | T / F | F

F → (E) | digit

Here:

E = addition/subtraction level

T = multiplication/division level

F = basic values

Since T must be reduced before E, multiplication is performed before addition.

Therefore:

8 + 2 * 4

becomes:

8 + (2 * 4)

= 8 + 8

= 16

The left-recursive productions:

E → E + T

T → T * F

also provide **left associativity**.

Conclusion

S-attributed definitions are suitable for bottom-up parsing because attributes can be evaluated when productions are reduced.

Q3. S-Attributed and L-Attributed Definitions

Given:

A → B C D

Attributes:

s = synthesized

i = inherited

Basic Rules

S-attributed definition

An S-attributed definition can contain **only synthesized attributes**.

Therefore, any use/definition of an inherited attribute means it is **not S-attributed**.

L-attributed definition

For:

A → B C D

an inherited attribute of:

- B can depend on attributes of A.
- C can depend on A and attributes of B.
- D can depend on A, B, and C.

Synthesized attributes can be evaluated from the children.

Set (i)

A.s = B.i + C.i

S-attributed?

No.

Reason:

B.i

C.i

are inherited attributes.

S-attributed definitions allow only synthesized attributes.

L-attributed?

Yes, structurally, but incomplete.

The rule defines a synthesized attribute of A. It does not introduce an illegal dependency for an inherited attribute.

However, B.i and C.i must be defined elsewhere.

Circular dependency?

No circular dependency is present in the given rule.

Result

S-attributed: No

L-attributed: Yes, as written/incomplete

Circular dependency: No

Set (ii)

Rules:

A.s = B.i + C.s

D.i = A.i + B.s

S-attributed?

No.

Reason:

B.i

D.i

A.i

are inherited attributes.

L-attributed?

Yes.

For:

D.i = A.i + B.s

D is the third symbol in:

A → B C D

and its inherited attribute depends on:

- A.i – parent inherited attribute
- B.s – left sibling synthesized attribute

Both are allowed in an L-attributed definition.

Circular dependency?

There is no direct cycle in the given rules.

Evaluation can proceed as:

A.i and B.s

↓

D.i

B.i and C.s

↓

A.s

Result

S-attributed: No

L-attributed: Yes

Circular dependency: No

Set (iii)

Rule:

A.s = B.s + D.s

S-attributed?

Yes.

Only synthesized attributes are used:

A.s

B.s

D.s

Therefore, it satisfies the definition of an S-attributed SDD.

L-attributed?

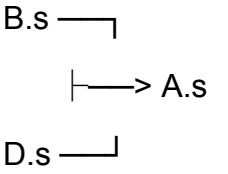
Yes.

Every S-attributed definition is also L-attributed.

Circular dependency?

No.

The dependency is:



There is no dependency back to B.s or D.s.

Result

S-attributed: Yes

L-attributed: Yes

Circular dependency: No

Set (iv)

Rules:

A.s = D.i

B.i = A.s + D.s

C.i = B.s

D.i = B.i + C.i

S-attributed?

No.

The rules contain inherited attributes:

B.i

C.i

D.i

Therefore, it is not S-attributed.

L-attributed?

No.

The problem is:

B.i = A.s + D.s

B.i is an inherited attribute of the first child B.

For an L-attributed definition, B.i can depend on attributes of A, but it cannot depend on the synthesized attribute D.s of a right sibling.

Therefore, this violates the L-attributed condition.

Circular Dependency

The dependency chain is:

A.s

↓

B.i

↓

D.i

↓

A.s

More clearly:

A.s = D.i

D.i = B.i + C.i

B.i = A.s + D.s

Therefore:

A.s → B.i → D.i → A.s

This is a **circular dependency**.

There is also:

C.i = B.s

but this does not remove the circular dependency.

Result

S-attributed: No

L-attributed: No

Circular dependency: Yes

Final Comparison Table

Set	S-attributed	L-attributed	Circular?	Reason
(i)	No	Yes*	No	Uses inherited attributes
(ii)	No	Yes	No	D.i depends on parent and left sibling
(iii)	Yes	Yes	No	Only synthesized attributes
(iv)	No	No	Yes	A.s → B.i → D.i → A.s

* Set (i) is structurally L-attributed as written, but the inherited attributes B.i and C.i have no defining rules, so the SDD is incomplete.

Conclusion

An attribute grammar can be evaluated only when its dependency graph has no cycle. A circular dependency prevents the compiler from deciding which attribute should be evaluated first.

Q4. Type Propagation in Variable Declaration

Given grammar:

D → T L

T → int

T → real

L → L , id

L → id

Input:

real p, q, r

The objective is to assign:

p → real

q → real

r → real

(i) Syntax-Directed Definition

Use:

T.type

L.in

id.type

where:

- T.type = synthesized type
- L.in = inherited type
- id.type = type assigned to identifier

Semantic Rules

D → T L

L.in = T.type

T → int

T.type = int

T → real

T.type = real

For the identifier list:

L → L1 , id

L1.in = L.in

id.type = L.in

For a single identifier:

L → id

id.type = L.in

Also update the symbol table:

insert(id.name, id.type)

(ii) Attributes Used

1. T.type

This is a **synthesized attribute**.

It obtains the type from:

int

or:

real

For:

real

we get:

T.type = real

2. L.in

This is an **inherited attribute**.

It carries the type from T to the identifier list.

T.type → L.in

3. id.type

This stores the type of the identifier.

For example:

p.type = real

q.type = real

r.type = real

(iii) Bottom-Up Evaluation

Input:

real p, q, r

First:

T → real

Therefore:

T.type = real

Then:

D → T L

propagates:

L.in = T.type

= real

Now consider:

L → L , id

For:

real p, q, r

the identifier list is:

((p,q),r)

At the final list:

r.type = real

Then the previous list gets:

q.type = real

Finally:

p.type = real

Symbol Table

Identifier	**Type**
p	real
q	real
r	real

Important Note About Bottom-Up Evaluation

Inherited attributes normally flow from parent to child, so they are naturally associated with top-down/L-attributed evaluation.

In a bottom-up parser, the inherited value can be carried in the **parser stack/semantic-value stack** or implemented using semantic actions/marker information.

Thus, during reductions, the parser maintains:

L.in = real

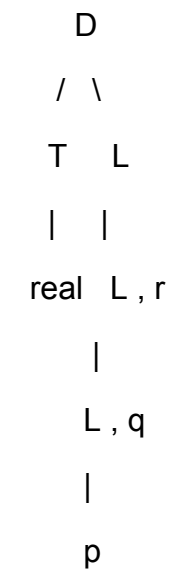
and uses it when reducing each identifier.

(iv) Annotated Parse Tree

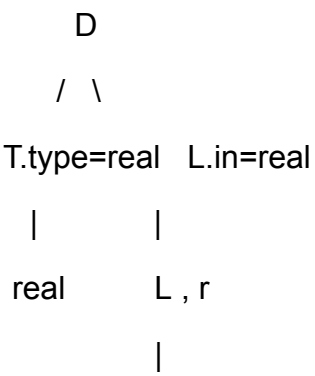
For:

real p, q, r

the parse tree can be shown as:



Annotated form:



L.in=real

|

L , q

|

L.in=real

|

p

Identifier attributes:

p.type = real

q.type = real

r.type = real

Therefore:

real p, q, r

is entered into the symbol table as:

p : real

q : real

r : real

(v) Bottom-Up Parser Stack

A simplified stack demonstration is:

Step	Stack	Remaining Input	Action	Attribute Evaluation
1	real	p , q , r	Shift	—
2	T	p , q , r	Reduce T→real	T.type=real
3	T p	, q , r	Shift	—
4	T L	, q , r	Reduce L→id	L.in=real, p.type=real

Step	Stack	Remaining Input	Action	Attribute Evaluation
5	T L ,	q , r	Shift	—
6	T L , q	, r	Shift	—
7	T L	, r	Reduce L→L,id	q.type=real
8	T L ,	r	Shift	—
9	T L , r	End	Shift	—
10	T L	End	Reduce L→L,id	r.type=real
11	D	End	Reduce D→T L	L.in=T.type=real

Final symbol table:

p → real

q → real

r → real

Conclusion

The type real is first obtained from T. It is then propagated to the complete identifier list, so every identifier receives the correct type.

Q5. Top-Down Translation Using L-Attributed Definition

Given grammar:

S → E n

E → E + T | T

T → T * F | F

F → (E) | digit

Input:

3 * 4 + 6 n

Important Point

The given grammar is **left recursive**:

E → E + T

T → T * F

A direct top-down parser cannot use left-recursive productions because they can cause infinite recursion.

Therefore, for top-down parsing, eliminate left recursion.

Transformed Grammar

S → E n

E → T E'

E' → + T E' | ε

T → F T'

T' → * F T' | ε

F → (E) | digit

This grammar is suitable for top-down parsing.

(i) L-Attributed SDD

Use synthesized attribute:

E.val

T.val

F.val

For the remaining expression, use inherited attribute:

E'.inh

T'.inh

Production 1

S → E n

Semantic rule:

S.val = E.val

Production 2

E → T E'

Semantic rules:

E'.inh = T.val

E.val = E'.syn

Production 3

E' → + T E'1

Semantic rules:

E'1.inh = E'.inh + T.val

E'.syn = E'1.syn

Production 4

E' → ε

Semantic rule:

E'.syn = E'.inh

Production 5

T → F T'

Semantic rules:

T'.inh = F.val

T.val = T'.syn

Production 6

T' → * F T'1

Semantic rules:

T'1.inh = T'.inh * F.val

T'.syn = T'1.syn

Production 7

T' → ε

Semantic rule:

T'.syn = T'.inh

Production 8

F → (E)

Semantic rule:

F.val = E.val

Production 9

F → digit

Semantic rule:

F.val = digit.lexval

(ii) Attribute Propagation

Input:

3 * 4 + 6

First:

3

is read.

Therefore:

F.val = 3

Then:

T'.inh = 3

Next:

* 4

is processed.

F.val = 4

Therefore:

T'1.inh = 3 * 4

= 12

After * operation:

T.val = 12

Now process:

+ 6

Since:

E'.inh = T.val = 12

and:

F.val = 6

we obtain:

E'1.inh = 12 + 6

= 18

Finally:

E.val = 18

(iii) Top-Down Translation

Input:

3 * 4 + 6 n

Step 1

E → T E'

Process T.

T → F T'

Process:

F → digit

So:

F.val = 3

Therefore:

T'.inh = 3

Step 2

Read:

*

Then:

T' → * F T'

Read 4.

F.val = 4

Therefore:

T'1.inh = T'.inh * F.val

= 3 * 4

= 12

Then:

T.val = 12

Step 3

Now read:

+

We use:

E' → + T E'

The current value is:

E'.inh = 12

Process T.

F → digit

For 6:

F.val = 6

Since there is no multiplication after 6:

T.val = 6

Step 4

Addition:

E'1.inh = 12 + 6

= 18

At the end:

E'.syn = 18

Therefore:

E.val = 18

Finally:

S.val = 18

Final Answer

3 * 4 + 6

= 12 + 6

= 18

(iv) Annotated Parse Tree

The important expression tree is:

```
      +
    /   \
   *     6
```

```
      /   \
     3     4
```

Annotated values:

```
          + [18]
        /       \
     * [12]      6 [6]
    /      \
 3 [3]     4 [4]
```

Therefore:

3 * 4 = 12

12 + 6 = 18

Final:

E.val = 18

Attribute Flow

The flow can be summarized as:

```
        E
       / \
      T   E'
      |   |
     12   inh=12
          |
          + 6
          |
         inh=18
          |
         syn=18
```

For multiplication:

T

|

F = 3

|

T'.inh = 3

|

* 4

|

T'.inh = 3*4 = 12

|

T.val = 12

For addition:

E'.inh = 12

|

+ 6

|

E'.syn = 18

|

E.val = 18

1. Left-recursive grammar is suitable for bottom-up parsing but must be transformed for normal top-down parsing.
2. In Q5, left recursion is removed before applying top-down L-attributed translation.